

Documentação da Plataforma VNH

Hype App · Você no Hype

Visão geral, arquitetura, dados, módulos, portal do cliente, links externos, comunicação e design

Versão 1.297.0 · outubro de 2026

- 1. 01 · Visão geral
- 2. 02 · Arquitetura (dev)
- 3. 03 · Dados e segurança
- 4. 04 · Módulos do app interno
- 5. 05 · Portal do cliente, links externos e integrações
- 6. 06 · Como as partes se comunicam e como as funcionalidades se ligam
- 7. 07 · Design
- 8. 08 · Lacunas, pendências e riscos

01 · Visão geral

O que é

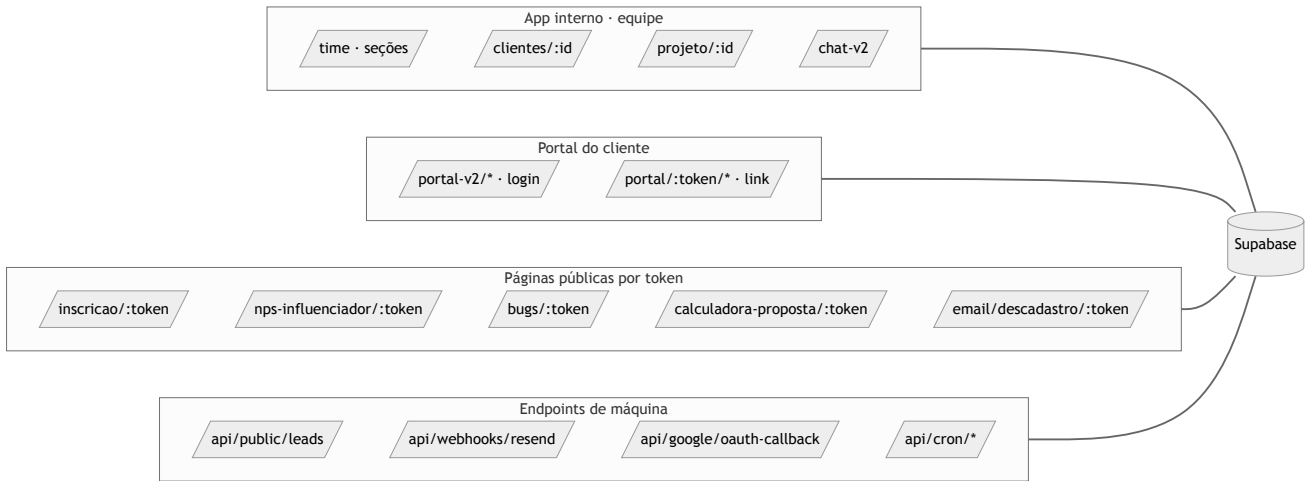
Plataforma de operação da agência **Você no Hype** (marketing de influência). Reúne, num só sistema, o que a equipe usa no dia a dia (comercial, clientes, campanhas, influenciadores, projetos e tarefas, reuniões, financeiro, metas, chat, time) e o que o **cliente** e o **influenciador** acessam por fora (portal, páginas públicas por link).

Construída no **Lovable** e conectada a um backend **Supabase** (Lovable Cloud). Produção em <https://plataforma.vocenhype.com.br> (Vercel).

Quem usa e por onde entra

Pessoa	Superfície	Como autentica
Equipe interna (admin ou membro com permissões)	App interno <code>/time</code> , <code>/clientes/\$id</code> , <code>/projeto/\$id</code> , <code>/chat-v2</code>	E-mail + senha (Supabase Auth), MFA se ativado
Cliente com login (empresa-cliente)	Portal do cliente V2 <code>/portal-v2/*</code>	E-mail + senha por convite; escopo por organização e por campanha
Cliente por link (sem login)	Portal por token <code>/portal/\$token/*</code>	Token público fixo do cliente
Influenciador / lead / terceiro	Páginas públicas: inscrição, NPS, bugs, proposta, descadastro	Token na URL
Sistemas externos	Webhooks e crons: leads (Make/Typeform), Resend, Google, Vercel Cron	Segredo/assinatura/Bearer

As cinco superfícies



Glossário

- Cliente** — empresa atendida pela agência. Cada cliente tem uma **organização** dedicada (para o portal).
- Campanha** — ação de influência de um cliente (briefing, orçamento, influenciadores, entregas, resultados).
- Influenciador (influ)** — criador de conteúdo; existe no **Banco de Influenciadores** e é vinculado a campanhas.
- Entrega** — conteúdo que o influenciador produz (roteiro → conteúdo → publicação), com versões e aprovação do cliente.
- Lead / oportunidade** — registro do Comercial (pipeline); vira Cliente + Projeto na conversão.
- Projeto** — trabalho interno (tarefas, marcos, bugs) ligado a um cliente.
- Hypito** — assistente da plataforma. Aparece em `hypito_*` (tabelas), `hypito_payload` nas mensagens do chat e em permissões. O serviço em si **não está neste repositório** [inferido: roda fora, usa as mesmas tabelas].
- Ambiente** — a organização em que a pessoa entra: `internal` (equipe) ou `client`.
- Seção** — módulo dentro de `/time` (`?section=`), não é rota separada.

02 · Arquitetura (dev)

Stack

Camada	Tecnologia
Framework	TanStack Start (React 19 + TanStack Router, SSR via Nitro/h3), file-based routing
Build	Vite 8, TypeScript estrito, gerenciador bun (<code>bun.lock</code> , <code>bunfig.toml</code>)
UI	Tailwind v4, shadcn/ui (Radix, estilo "new-york"), lucide-react, TipTap (editor rico)
Dados	Supabase (Postgres + Auth + Realtime + Storage), <code>@supabase/supabase-js</code> , TanStack Query
Deploy	Vercel (<code>vercel.json</code> : headers de segurança + 2 crons); código sincroniza com Lovable pelo branch git
E-mail	Resend (provedor configurável por admin; webhook de entrega)
Outros	Web Push (VAPID), Google Calendar (OAuth), Open-Meteo (clima), WebRTC (chamadas, TURN opcional)

Regra de repositório (AGENTS.md): nunca reescrever histórico já publicado (force-push, rebase/amend/squash) — dessincroniza o Lovable. Arquivos `src/integrations/supabase/*` são gerados.

Comandos

```
bun install
bun run dev          # http://localhost:8080
bun run build
bun run typecheck    # tsc --noEmit
bun run lint
bun run test          # vitest (65 arquivos, 808 testes na última verificação)
bun scripts/release.ts bump|publish  # versão + aviso em tempo real
```

Nota: `vitest` deve ignorar `.claude/worktrees/**` (cópias antigas fazem testes falharem).

Estrutura de pastas

```
src/
  routes/           rotas (file-based). routeTree.gen.ts é gerado
  components/       telas e peças do app interno (uma *Section por módulo) + ui/ (shadcn) + shared/ (canônicos)
  features/client-portal-v2/  o Portal V2 (layout, páginas, componentes, lib, testes)
  lib/              regra de negócio, stores, server functions (*.functions.ts), engines puros
  hooks/            use-confirm, use-mobile, use-weather, ...
  integrations/supabase/  client, client.server (service-role), auth-middleware, types (gerados)
  styles.css styles/  tokens (oklch) e utilidades
supabase/migrations/ 144 arquivos SQL, timestamp-nomeados, aditivos
public/              PWA (manifest, sw.js sem cache), version.json, sons
docs/                esta documentação + design-system + auditorias
scripts/release.ts   release local (não há CI)
```

Rotas

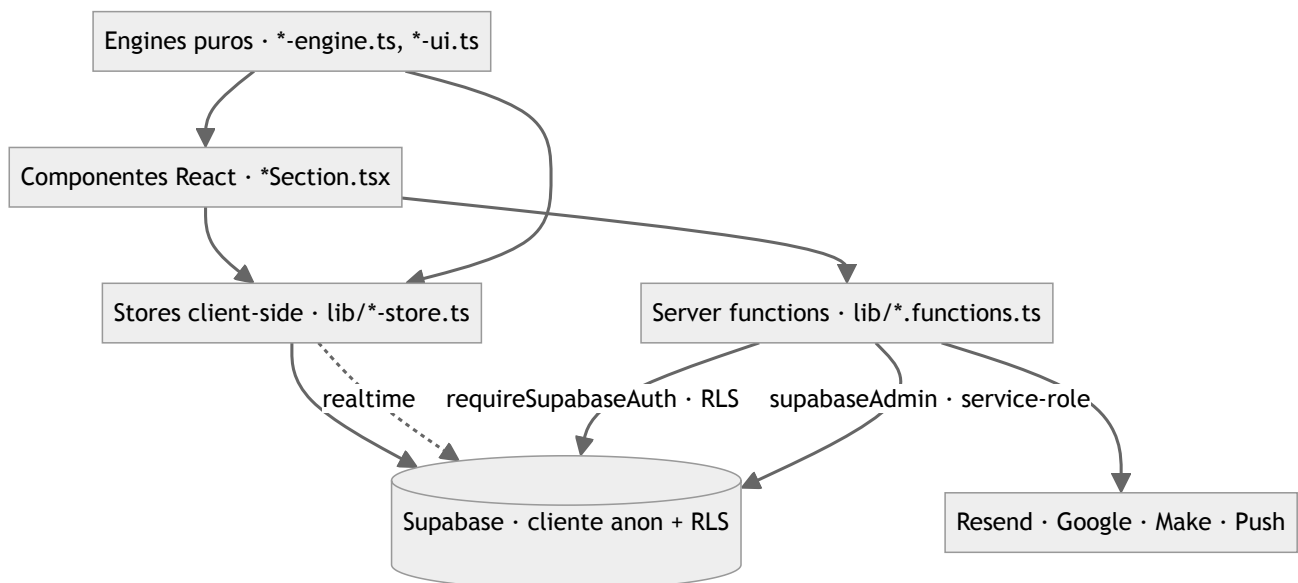
Layout autenticado `/_authenticated/route.tsx` : confere sessão (redireciona para `/`), `profiles.must_change_password` (→ `/primeiro-acesso`), inicializa stores/sync e a lista do time (poll 30s).

Rota	Tipo	Função
<code>/</code>	pública	Login (<code>signInWithPassword</code>)
<code>/_authenticated/time</code>	app	Home do app: AppShell + seção por <code>?section=</code> (início, clientes, campanhas, projetos, reunioes, comercial, financeiro, time, influenciadores, metas, chat, configuracoes, problemas) e evento <code>nav:section</code>

Rota	Tipo	Função
/clientes/\$id , /projeto/\$id	app	Detalhe de cliente / projeto (únicas seções com rota própria, além do chat)
/chat-v2 (+ channel/\$id , dm/\$id , campaign/\$id)	app	Chat V2 dentro do AppShell
/design-system , /design-system-finance-concept	app	Vitrine interna do design system
/primeiro-acesso , /criar-senha , /redefinir-senha	auth	Troca forçada (equipe), convite (cliente), recuperação
/selecionar-ambiente , /acesso-pendente , /acesso-bloqueado	auth	Escolha de organização, convite pendente, organização suspensa
/portal-v2/*	cliente	Portal com login (ver 05)
/portal/\$token/*	cliente	Portal por link (ver 05)
/portal-app/*	legado	Redirects para /portal-v2
/inscricao/\$token , /nps-influenciador/\$token , /bugs/\$token , /calculadora-proposta/\$token , /email/descadastro/\$token	pública	Páginas por token (ver 05)
/api/public/leads , /api/webhooks/resend , /api/google/oauth-callback , /api/cron/email-flows , /api/cron/google-calendar-sync	máquina	Endpoints (ver 05)
/foco , /time-v2 , /banco-influenciadores-v2	redirect	Compatibilidade (cutover concluído)

Observação importante: a maior parte do app interno é **uma SPA dentro de /time** , trocada por `?section=` — não são rotas TanStack separadas. `src/lib/section-nav.ts` é a fonte dos tipos e rótulos de seção/aba.

Camadas



- UI** — uma `*Section.tsx` por módulo; subcomponentes na pasta do módulo.
- Engines/regra pura** — `entrega-engine`, `metas-engine`, `comercial-engine`, `performance-engine`, `insights-engine`, `aeo-engine` e `*-ui.ts`: sem I/O, testáveis; a UI nunca recalcula saúde/progresso na mão.
- Stores** — pub/sub escritos à mão (sem Redux/Zustand): `createTableArrayStore` (uma linha por entidade em `clientes`, `projetos`, `reunioes`, `financeiro_lancamentos`, `banco_influenciadores`, `marketing_tasks`, `metas`, `aeo_*`...), mais `chat-store`, `workspace-store`, `shared-sync` (`shared_state`), `campanha-scoped-store`, `projeto-scoped-store`. Mantém contrato síncrono `get/set/subscribe`; `init()` roda uma vez no `beforeLoad` do layout autenticado.
- Server functions** (`createServerFn`) — 34 arquivos `*.functions.ts`. O token do usuário é anexado por `auth-attacher` e validado por `requireSupabaseAuth` (injeta `supabase`, `userId`, `claims`).
- Admin/service-role** — só em `*.server.ts` ou via `await import(...)` dentro de handlers; todo uso de `supabaseAdmin` passa por checagem explícita (`assertAdmin` / `is_admin()` ou validação de token).

Autenticação e ambientes

- Login por senha; `must_change_password` força `/primeiro-acesso` .
- **MFA** (TOTP) via `mfa.functions.ts` ; o portal e o app redirecionam para `/` se o nível de garantia exigir desafio.
- **Ambiente** (`user-environment.server.ts`): lê `organization_members` ativos em `organizations` ativas e devolve `internal` , `client` , `multiple` (→ `/selecionar-ambiente`), `pending` (→ `/acesso-pendente`) ou `suspended` (→ `/acesso-bloqueado`).
Home do cliente: `/portal-v2/inicio` ; da equipe: `/time` .
- **Convites** (`organization-invites` , `accept-invite`): aceitam convites pendentes ao logar.
- **Recuperação de senha** com rate limit (`rate-limit.functions.ts`) antes de existir sessão.
- **Cofre de senhas** (`vault*.functions.ts`) com TOTP próprio e pedidos de acesso.

Variáveis de ambiente

Variável	Uso
<code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_PUBLISHABLE_KEY</code> (+ <code>SUPABASE_*</code> no servidor)	cliente Supabase
<code>SUPABASE_SERVICE_ROLE_KEY</code>	só servidor ; cliente admin. Não existe localmente por padrão
<code>APP_URL</code>	origem canônica (redirect do OAuth Google); nunca derivada do request
<code>SITE_URL</code> , <code>VERCEL_URL</code> , <code>VERCEL_ENV</code>	origem de links e ambiente
<code>GOOGLE_CLIENT_ID</code> , <code>GOOGLE_CLIENT_SECRET</code>	OAuth do Google Calendar
<code>CRON_SECRET</code>	Bearer exigido pelos crons
<code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code> , <code>VITE_VAPID_PUBLIC_KEY</code>	Web Push
<code>BLOG_WEBHOOK_URL</code>	webhook de blog (Make)
<code>VITE_TURN_URLS</code> , <code>VITE_TURN_USERNAME</code> , <code>VITE_TURN_CREDENTIAL</code>	TURN para chamadas
O segredo do webhook de leads não é variável: fica em <code>webhook_settings</code> (só <code>service-role</code>).	

Build, deploy, versão

- Deploy por observação do branch (Lovable Cloud) / Vercel; **não há CI**.
- `public/version.json` + `VersionWatcher` avisam o usuário quando há bundle novo; `APP_VERSION` em `lib/app-version.ts` .
- `scripts/release.ts` : `bump` (SemVer, roda tsc/eslint/testes, commita) e `publish` (insere em `platform_releases` , disparando aviso em tempo real; o Hypito anuncia).
- Service worker (`sw.js`) existe só para instalação como PWA e **não faz cache**.

Convenções de código

Alias `@/*` → `src/*` ; Prettier 100 colunas, aspas duplas, vírgulas finais; `server-only` do Next é bloqueado (usar `*.server.ts`); migrations novas como arquivos novos, nunca editar antigas.

03 · Dados e segurança

Backend: Supabase projeto `kehjxyzrolltsdaqzkww` . 144 migrations aditivas em `supabase/migrations/` . RLS ligada em todas as tabelas de domínio.

Modelo de dados por domínio

Domínio	Tabelas principais	Observação
Identidade e acesso	<code>profiles</code> , <code>user_roles</code> , <code>organizations</code> , <code>organization_members</code> , <code>campaign_members</code> , <code>access_audit_log</code> , <code>settings_audit_log</code> , <code>password_reset_requests</code> , <code>rate_limit_events</code>	<code>profiles.permissions</code> guarda as permissões do membro; <code>organizations.type</code> é <code>internal</code> ou <code>client</code>
Clientes e contratos	<code>clientes</code> , <code>contratos</code>	<code>clientes.data</code> é JSONB e embute as campanhas do cliente
Comercial	<code>leads</code> , <code>commercial_interactions</code> , <code>comercial_saved_views</code> , <code>pricing_settings</code>	Leads vêm do webhook ou da UI; <code>proposta</code> pública por token
Campanhas	<code>campanha_influenciadores</code> , <code>campanha_tarefas</code> , <code>campanha_documentos</code> , <code>campanha_cronograma</code> , <code>campanha_entregas</code> , <code>campanha_entrega_versoes</code> , <code>campanha_entrega_eventos</code> , <code>campanha_nps</code> , <code>campanha_nps_influenciador</code> , <code>campanha_influenciador_avaliacoes</code> , <code>campaign_cycles</code> , <code>inscricao_campanha_idempotency</code> , <code>influencer_approvals</code>	Tabelas "scoped": cada linha tem <code>campanha_id</code> ; o dado completo fica em <code>data</code> JSONB com <code>data.id</code> sincronizado por trigger
Influenciadores	<code>banco_influenciadores</code> , <code>projeto_influenciadores</code>	Dados bancários exigem permissão própria
Projetos e tarefas	<code>projetos</code> , <code>projeto_tarefas</code> , <code>task_blocks</code> , <code>task_dependencies</code> , <code>task_tags</code> , <code>marketing_tasks</code> , <code>marketing_standalone_tasks</code> , <code>time_entries</code>	Tarefas também existem dentro de campanhas e do Marketing; <code>task-aggregation</code> as une
Reuniões	<code>reunioes</code> , <code>reunioes_disponibilidade</code> , <code>google_calendar_connections</code> , <code>google_calendar_sync_state</code> , <code>google_oauth_states</code> , <code>shared_calendar_connection</code>	Sync opcional com Google Calendar
Financeiro	<code>financeiro_lancamentos</code> , <code>financeiro_recorrencias</code> , <code>financeiro_settings</code> , <code>financeiro_status_overrides</code>	Lançamentos podem apontar para cliente, campanha e influenciador
Metas	<code>metas</code> , <code>performance_events</code> , <code>performance_settings</code>	Objetivos e indicadores; motor de saúde puro em <code>metas-engine</code>
Chat	<code>chat_channels</code> , <code>chat_messages</code> , <code>chat_reads</code> , <code>chat_status</code> , <code>chat_conversations</code> , <code>chat_conversation_members</code> , <code>chat_message_attachments</code> , <code>chat_message_reactions</code> , <code>chat_pinned_messages</code> , <code>chat_saved_messages</code> , <code>chat_deliveries</code> , <code>chat_drafts</code>	Convive V1 (<code>/time?section=chat</code>) e V2 (<code>/chat-v2</code>); trigger sincroniza JSONB legado para tabelas
Marketing	<code>email_campaigns</code> , <code>email_campaign_steps</code> , <code>email_campaign_recipients</code> , <code>email_campaign_activity</code> , <code>email_flows</code> , <code>email_flow_enrollments</code> , <code>email_sends</code> , <code>email_templates</code> , <code>email_unsubscribes</code> , <code>email_provider_settings</code> , <code>aeo_prompts</code> , <code>aeo_rodadas</code> , <code>aeo_respostas</code> , <code>blog_comments</code> , <code>blog_likes</code>	E-mail (Resend), monitor AEO e blog
Central de Problemas	<code>bug_reports</code> , <code>bug_report_comments</code> , <code>bug_report_events</code> , <code>bug_report_attachments</code> , <code>bug_report_diagnostics</code>	Triagem exige <code>can_manage_problems</code>
Hypito	<code>hypito_action_log</code> , <code>hypito_alerts_sent</code> , <code>hypito_conversation_state</code> , <code>hypito_daily_briefing_runs</code> , <code>hypito_meeting_reminders_sent</code> , <code>hypito_pending_actions</code> , <code>hypito_reminders</code> , <code>hypito_report_runs</code> , <code>hypito_report_settings</code> , <code>hypito_user_prefs</code>	Assistente/automações
Plataforma	<code>workspace_settings</code> , <code>shared_state</code> , <code>platform_releases</code> , <code>outgoing_webhooks</code> , <code>webhook_settings</code> , <code>push_subscriptions</code> , <code>personal_reminders</code>	Config global e avisos de versão
Segurança	<code>vault_secret</code> , <code>vault_totp_secrets</code> , <code>vault_totp_attempts</code> , <code>vault_access_requests</code>	Cofre de senhas

Domínio	Tabelas principais	Observação
Extras	daily_game_sessions(+archive) , sudoku_daily_results , zip_daily_results	Jogos diários

(projeto_fases foi removida na migration 20261003120000 .)

Storage (buckets)

avatars , chat-attachments , bug-reports , task-attachments , financeiro-anexos , entrega-anexos , relatorios-mensais , aeo-evidencias (5 receberam limite de 100 MB na remediação de setembro). Anexos públicos de bug-report por link vão em public/<token>/... . Relatórios mensais são entregues por URL assinada fresca (getFreshRelatorioUrl*).

Autorização em camadas

1. **RLS no banco** (a barreira real). Funções-chave: is_admin() , has_permission(uid, perm) (um admin + permissão), can_manage_problems , is_internal_team_member , shares_internal_organization , is_active_client_admin_of , user_can_access_campanha .
2. **Permissões de módulo** (lib/permissions.ts): clientes, campanhas, contratos, projetos, reunioes, comercial, financeiro, time, influenciadores, influenciadores:bancario, metas, chat, senhas, configuracoes (+ :perfil/:workspace/:av/:senhas), membros, problemas . Todas as tabelas de domínio chamam has_permission em suas policies.
3. **Server functions**: RLS-scoped (requireSupabaseAuth) por padrão; as que usam service-role validam admin ou token explicitamente.
4. **UI**: esconde/esmaece o que a pessoa não pode usar — conveniência, nunca a barreira.

Limitações aceitas e documentadas: - clientes aceita clientes ou campanhas , porque a campanha vive no JSONB do cliente. - membros é **decorativa**: criar/editar/excluir membro e redefinir senha seguem exigindo admin (um endpoint único altera role , e liberar abriria escalada de privilégio). - senhas é decorativa: o acesso real ao cofre é configuracoes:senhas .

Portal do cliente — isolamento

- Cliente com login: papéis client_standard (administra pessoas e acessos), client_approver , client_viewer ; **nunca** pode alterar as próprias permissões (prevent_client_permissions); proteção contra remover o último admin; acesso restrito a campanhas liberadas (campaign_members , user_can_access_campanha).
- Cliente por token: o token público do cliente (clientes.data.publicToken) é a credencial; as funções do cliente-link validam o token e escopam tudo ao cliente.

Outras proteções

- **MFA/TOTP**; rate limit de login e recuperação; X-Frame-Options , nosniff , Referrer-Policy (Vercel).
- Webhook de leads com X-Webhook-Secret (segredo em webhook_settings , rotacionável por admin); Resend com assinatura Svix; crons com Authorization: Bearer \$CRON_SECRET .
- Markdown do blog sanitizado (DOMPurify). Páginas por token têm noindex , nofollow .
- Relatórios completos: docs/security-audit-report.md e docs/security-remediation-plan.md .

04 · Módulos do app interno

Todos vivem sob `/time?section=<chave>` (exceto detalhes de cliente e projeto e o Chat V2). Cada módulo = uma `*Section.tsx` em `src/components/` + uma pasta de apoio. Permissão entre parênteses.

Seção	Função em uma linha
<code>inicio</code>	Painel pessoal: trabalho, agenda, mural, comentários, lembretes, clima
<code>clientes</code> (clientes)	Cadastro e ciclo de vida dos clientes
<code>campanhas</code> (campanhas)	Operação das campanhas, influenciadores e entregas
<code>projetos</code> (projetos)	Projetos e tarefas internas
<code>reunioes</code> (reunioes)	Agenda, disponibilidade, solicitações, Google Calendar
<code>comercial</code> (comercial)	Pipeline de leads, propostas, e-mail marketing
<code>financeiro</code> (financeiro)	Lançamentos, posição, análises
<code>time</code> (time)	Pessoas, carga, resposta, score operacional
<code>influenciadores</code> (influenciadores)	Banco de influenciadores
<code>metas</code> (metas)	Objetivos e indicadores
<code>chat</code> (chat)	Mensagens, canais, chamadas
<code>configuracoes</code> (configuracoes)	Perfil, workspace, permissões, integrações, segurança
<code>problemas</code>	Central de Problemas (reportar e triar bugs)
Marketing (dentro de Comercial/Marketing)	E-mail, blog, AEO, tráfego pago, editorial

Início

`InicioDashboard.tsx` (+ `components/inicio/`). Cards: **Meu trabalho** (tarefas via `task-aggregation`), **Agenda**, **Mural**, **Comentários**, **Lembretes** (`reminders.functions`, `personal_reminders`), **Pausa rápida** (jogos), cabeçalho climático (`weather-*`, `Open-Meteo`), diálogo "Bom dia" e avisos de release. Preferências por usuário em tempo real. É a **origem da linguagem visual** do design system.

Clientes

`ClientesSection`, `clientes/ClienteCard` · `ClienteFiltersBar` · `ClienteFormSheet` · `ClienteDetailPage` · `ClienteContratosSection` · `ClienteFinancialSummary` · `PortalAccessSection` · `ClienteStatusControl`. - Status do cliente: Captação → Ativo → Encerrado → Arquivado (`validate_cliente_status`; só admin arquiva). - Criar cliente chama `createClienteComOrganizacao` (cria a **organização** do portal junto). - Detalhe: contratos (`contratos.functions`), resumo financeiro, **acessos ao portal** (convites, papéis, campanhas liberadas — `client-access.functions`, `organization-invites`). - Vem de: conversão de Lead ou "Importar do CRM".

Campanhas

`CampanhasSection` (~1.700 linhas, lista + detalhe), `campanhas/*` (card, filtros, ativação, NPS, **página de inscrição**), `influenciadores/InfluencerBoard` (kanban/curadoria de influenciadores na campanha), `lib/entrega-engine`, `campanha-status`, `campanha-aprovacao`. - A campanha mora dentro de `clientes.data.campanhas`; dados operacionais em tabelas `campanha_*`. - Fluxo do influenciador na campanha: inscrito → curadoria → enviado ao cliente → aprovado/recusado → entregas (roteiro/conteúdo/publicação) → resultados. - **NPS** do cliente (`campanha_nps`) e do influenciador (`campanha_nps_influenciador`, com avaliação interna). - **Inscrição pública** com token (`signupToken`) e perguntas personalizáveis.

Projetos e tarefas

`ProjetosSection`, `projeto.$id` (página única com seções por âncora), `tasks/TaskBoard` (workspace de tarefa: subtarefas, tags, dependências, bloqueios, tempo, comentários, anexos, rich-editor), `ProjectBugsPanel`. - Tarefas de **projeto**, **campanha** e **Marketing avulso** são tabelas diferentes agregadas em "Meu trabalho". - Bloqueio de tarefa com questionário (`task-blocks`); dependências não travam, só avisam. - Link público de bugs por projeto (`/bugs/$token`).

Reuniões

`ReunioesSection`, `meetings/*` (Agenda, Calendário, Disponibilidade, Solicitações, entrada por link), `UpcomingMeetingAlert`, `MeetingReminderToast`. Sincroniza com **Google Calendar** (`google-calendar.functions`, OAuth, cron diário). Pode anexar chamada interna (WebRTC).

Comercial

`ComercialSection`, `comercial/*` (`PipelineBoard`, `LeadDrawer`, `LeadCard`, `FollowUp`, `SimuladorPropostaDialog`, `convertLead`), `comercial-engine`, `pricing`. - Etapas de oportunidade; realtime sobre `leads` (AppShell notifica novo lead). - **Simulador de Proposta** (tiers x formatos) e **calculadora pública** por token. - `convertLeadToClienteEProjeto`: lead → Cliente (sempre em Captação) + Projeto; grava `crmLeadId`. - Ações automáticas por oportunidade (`runOpportunityAction`), visões salvas, interações.

Marketing

`MarketingSection` + `marketing/*`: **E-mail** (campanhas, passos, públicos, templates, fluxos automáticos, dashboard; provedor Resend), **Blog** (editor markdown, publicação agendada, webhook para o site via Make, curtidas e comentários), **AEO Monitor** (prompts, rodadas, respostas por IA, evidências), **Tráfego pago**, **Editorial**.

Financeiro

`FinanceiroSection` com 3 áreas: **Resumo** (posição, requer atenção), **Lançamentos** (entradas/saídas, filtros, importação CSV, cobrança, marcar como pago), **Análises** (fluxo de caixa, despesas, receita por cliente, resultado por campanha). Lançamentos ligam a cliente/campanha/influenciador; pagamentos de influenciador alimentam o financeiro; contratos geram alertas de vencimento.

Time

`TimeV2Page`, `time-v2/*`, `team/*`: tabela de membros, carga, perfil (comunicação, tempo de resposta, entregas da semana, insights) e **Score Operacional** (`performance-engine`, config em Configurações). Tempo de resposta vem do Chat (`get_member_response_time`).

Influenciadores

`influenciadores-v2/InfluencerBancoV2Page`: banco com filtros, card, drawer e avaliação; dados bancários só com `influenciadores:bancario`. Alimenta campanhas e e-mail marketing (picker).

Metas

`metas/*`: **Objetivos** com **Indicadores** (peso, histórico, evolução, linha esperada), hoje atualizados **manualmente** (`dataSource: "auto"` está reservado para o futuro). Saúde: saudável, atenção, em risco, atrasado, concluído, não iniciado, cancelado — calculada só por `metas-engine`.

Chat

V1 (`ChatSection`, `chat-store`) e V2 (`chat-v2/*`, rotas `/chat-v2`). Canais públicos/privados, DMs, **@menções** de pessoa, tarefa, projeto, campanha e cliente (`mention-kinds`), anexos, mensagens de voz, reações, fixadas/salvas, busca, threads, links com preview, **chamadas** (`call-controller`, WebRTC) e push.

Configurações

Seções: Geral/Workspace, Perfil, Preferências, Disponibilidade, Áudio e Vídeo, Segurança (MFA, cofre), **Time e permissões**, **Integrações** (webhook de leads, webhooks de saída, Google), Precificação, Score operacional, Dados e backup, **Auditoria**.

Central de Problemas

`problemas/*`: qualquer membro **reporta** (com contexto automático: tela, navegador, versão, captura) e comenta; quem tem `problemas` **tria** (status, prioridade, área, responsável, nota interna). Os reports do portal do cliente também chegam aqui.

Shell e transversais

AppShell : sidebar global (256/68px), topbar, busca global, notificações (menções, tarefas, mensagens), timer ativo, tema, aviso de versão.

Transversais: useConfirm , FilterToolbar , Kpi , PageHeader , EmptyState , notificações/push, release notes.

05 · Portal do cliente, links externos e integrações

Portal do cliente — três gerações

Rota	Estado	Acesso
/portal-v2/*	Atual, destino de todo login de cliente	Sessão Supabase + organização client ativa
/portal/\$token/*	Em uso, por link	Token público fixo do cliente, sem login
/portal-app/*	Legado: só redirects para /portal-v2	—

Portal V2 (features/client-portal-v2 , rotas portal-v2/*)

Layout PortalV2Shell . Guard (portal-v2/route.tsx): exige sessão, MFA resolvido, troca de senha se pendente, aceita convites pendentes, resolve o ambiente; e aplica o **gate de NPS** — se há NPS do mês pendente, bloqueia a navegação até responder (nps-guard).

Página	Função
Início	Resumo, atenção (o que espera o cliente), progresso das campanhas, atividade
Campanhas / \$id	Visão geral, creators, conteúdo, arquivos, timeline, resultados
Aprovações	Perfis de influenciador e entregas aguardando o cliente
Conteúdos, Arquivos, Relatórios	Entregas, anexos, relatórios mensais (URL assinada)
NPS	Avaliação mensal por campanha
Notificações, Perfil, Conta, Segurança	Conta e preferências
Configurações → Acessos	Administradores do cliente convidam e gerenciam pessoas (papéis e campanhas)

Ações do cliente (server functions portal-auth.functions , sufixo Session): responder influenciador (aprovar/recusar/reabrir), responder entrega, editar briefing/observações/anexos, comentar, enviar demanda, reportar bug (cai na Central de Problemas), enviar NPS, abrir relatório, curtir/comentar artigo do blog. Papéis: client_standard (admin), client_approver (aprova), client_viewer (lê).

Portal por token (portal.\$token/*)

Mesmo conteúdo essencial (início, campanhas, aprovações, relatórios, solicitações) sem login, via cliente-link.functions (as mesmas ações, sufixo sem Session). Visual próprio em components/portal/* . Idiomas via portal-i18n . noindex .

Links externos (por token, sem login)

URL	Quem usa	O que faz	Credencial
/inscricao/\$token	Influenciador	Inscrive-se numa campanha (redes, nicho, perguntas personalizadas, anexo); idempotente	signupToken da campanha
/nps-influenciador/\$token	Influenciador	Avalia a experiência (notas + "voltaria a trabalhar")	token em campanha_nps_influenciador
/bugs/\$token	Cliente/influenciador	Lista, reporta e marca bugs/sugestões do projeto (com print)	Token do projeto
/calculadora-proposta/\$token	Lead	Monta e salva uma proposta no simulador	Token do lead
/email/descadastro/\$token	Destinatário	Descadastro (entra em email_unsubscribes e cancela o destinatário em todas as campanhas)	unsubscribe_token do destinatário
/portal/\$token/*	Cliente	Portal sem login	Token do cliente
/criar-senha , /redefinir-senha	Cliente/equipe	Convite e recuperação	Link do Supabase Auth

Todas devolvem noindex , nofollow e escopam a leitura/escrita ao dono do token. Tentativas inválidas caem em estado de erro amigável.

Endpoints de máquina

Endpoint	Chamado por	Proteção	O que faz
POST /api/public/leads	Make / Typeform / formulários	X-Webhook-Secret (em webhook_settings); CORS	Insere o lead via service-role (o disparo lead.created / lead.won para webhooks de saída acontece em comercial.functions, ao criar/ganhar pela UI)
POST /api/webhooks/resend	Resend	Assinatura Svix (segredo whsec_ guardado pelo admin em email_provider_settings)	Atualiza o estado de cada envio em email_sends
GET /api/google/oauth-callback	Google	state em google_oauth_states; APP_URL canônico	Conclui OAuth do Calendar
GET /api/cron/email-flows	Vercel Cron 0 13 * * *	Bearer CRON_SECRET	Avança fluxos de e-mail automáticos
GET /api/cron/google-calendar-sync	Vercel Cron 0 6 * * *	Bearer CRON_SECRET	Sincroniza reuniões ↔ Google

Integrações de saída

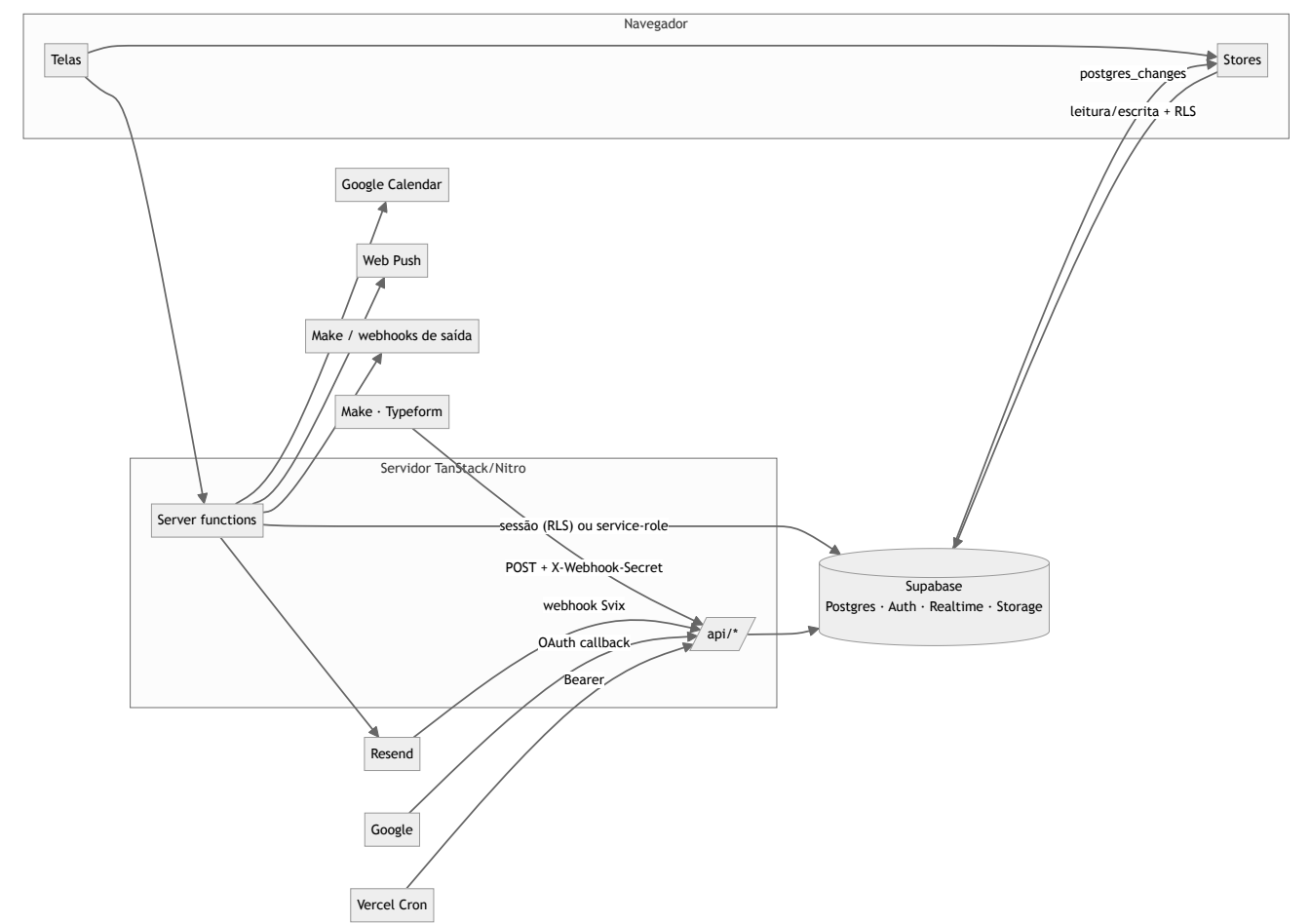
Destino	Como	Gatilho
Webhooks configuráveis (Make etc.)	outgoing_webhooks (admin, Configurações → Integrações), POST JSON {event,data,timestamp}; best-effort	lead.created, lead.won
Webhook do blog (Make)	BLOG_WEBHOOK_URL, campo action = upsert/archive/delete	Ciclo de vida do artigo
Resend	email-provider.server, configurado em email_provider_settings	Disparos de campanha e fluxos
Google Calendar	OAuth + googleapis.com/calendar/v3	Reuniões
Web Push	VAPID (push.functions, push_subscriptions)	Chat, menções, lembretes
Open-Meteo	weather-service	Cabeçalho da Início
WhatsApp (wa.me), redes sociais	Links gerados na UI	Contato e perfis

06 · Como as partes se comunicam e como as funcionalidades se ligam

1. Os canais de comunicação

Há **cinco** formas de uma parte falar com outra. Quase tudo passa pelo Supabase (banco como ponto de encontro); não há fila de mensagens nem API REST própria além dos endpoints `/api/*`.

Canal	Quem fala com quem	Como
Leitura/escrita direta	Navegador → Supabase	<code>supabase-js</code> com a sessão do usuário; RLS decide o que passa. É o caminho dos <code>stores</code> (<code>clientes</code> , <code>projetos</code> , <code>financeiro_lancamentos</code> , <code>chat...</code>)
Server functions	Navegador → servidor → Supabase/serviços	<code>useServerFn(...)</code> ; token do usuário anexado automaticamente; usadas quando precisa de segredo, <code>service-role</code> , e-mail, regra sensível ou token público
Realtime	Supabase → navegador	<code>postgres_changes</code> em canais nomeados (<code>rt-*</code>); atualiza caches e dispara notificações
Webhooks/Endpoints	Sistema externo ↔ <code>/api/*</code>	Entrada: leads, Resend, Google. Saída: webhooks de leads e blog
Agendados	Vercel Cron → <code>/api/cron/*</code>	E-mail flows e Google Calendar, uma vez por dia

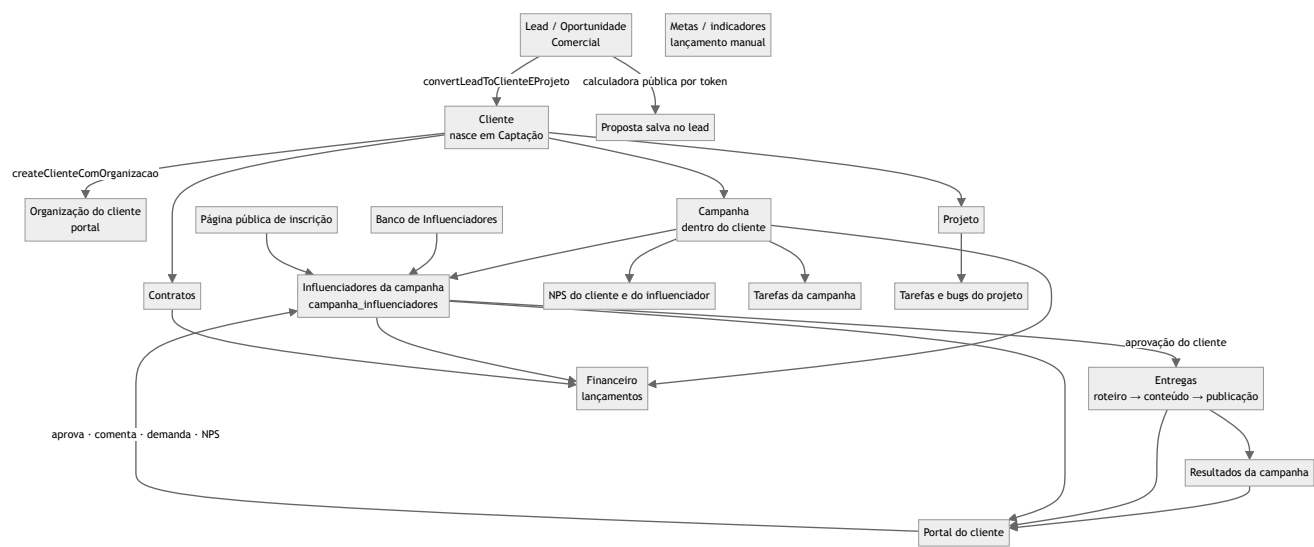


Tempo real: o que escuta o quê

Canal Realtime	Tabela / evento	Reação
<code>rt-leads-*</code> (Comercial)	<code>leads</code> *	Recarrega o pipeline
<code>rt-nav-leads-*</code> (AppShell)	<code>leads</code> INSERT	Notificação de novo lead

Canal Realtime	Tabela / evento	Reação
rt-campanha-influenciadores-notify	campanha_influenciadores UPDATE	Ação do cliente (aprovou/recusou/comentou) vira item no sino da equipe
rt-campanha-tarefas-demand-notify	campanha_tarefas INSERT	Demanda enviada pelo cliente vira notificação + toast
rt-chat-channels , rt-chat-messages , rt-chat-status	tabelas de chat	Mensagens, status, não lidas, menções
rt-portal-campanha-influenciadores , rt-portal-app-...	campanha_influenciadores	O portal do cliente reflete mudanças da equipe
shared_state:changes	shared_state	Estado compartilhado entre abas e pessoas
rt-dashboard-prefs-* , rt-bomdia-*	prefs/briefing	Início e "Bom dia"
rt-<tabela>-* (stores de lista)	clientes , projetos , reunioes , financeiro_lancamentos ...	Cache local sincronizado entre usuários
O cache local também guarda cópia em <code>localStorage</code> e o estado do time é consultado a cada 30 s.		

2. A cadeia principal do negócio



Leitura da cadeia: 1. **Lead** → **Cliente**. `convertLeadToClienteEProjeto` cria Cliente (sempre **Captação**, nunca Ativo direto), grava `crmLeadId` e cria um Projeto. A ativação é uma ação explícita separada (`ClienteStatusControl`). 2. **Cliente** → **organização**. Todo cliente novo ganha organização própria; é ela que dá acesso ao Portal V2 e isola dados. 3. **Cliente** → **campanha** → **influenciadores**. Influenciadores entram na campanha pelo **Banco**, por **inscrição pública** ou manualmente; passam por curadoria interna e vão ao cliente. 4. **Aprovação no portal**. O cliente aprova/recusa perfis e entregas; isso volta à equipe em tempo real (sino). 5. **Entregas** → **resultados**. Estado e métricas das entregas alimentam a aba de resultados da campanha. O **relatório mensal** é um arquivo guardado no bucket `relatorios-mensais` e entregue ao cliente por URL assinada (não é gerado a partir das entregas). 6. **Dinheiro**. Lançamentos podem referenciar cliente, campanha e influenciador; contratos geram alerta de vencimento; o financeiro por campanha compõe Análises. 7. **Metas** hoje são alimentadas por **lançamento manual** de indicadores (a fonte `auto` existe no modelo, mas está reservada para o futuro) e produzem saúde por `metas-engine` ; o **Time** consome tarefas, entregas e chat para carga, resposta e score.

3. Funcionalidades correlatas (quem toca em quem)

Funcionalidade	Está ligada a	Natureza do vínculo
Clientes	Comercial, Campanhas, Contratos, Financeiro, Portal, Chat (@cliente)	Conversão de lead; campanhas embutidas; organização; menções
Campanhas	Influenciadores (banco + inscrição), Entregas, NPS, Tarefas, Financeiro, Portal, Chat (@campanha, canal por campanha)	Mesmo <code>campanha_id</code> em tabelas <code>campanha_*</code>

Funcionalidade	Está ligada a	Natureza do vínculo
Banco de Influenciadores	Campanhas, Financeiro (pagamento), E-mail marketing (picker), Avaliações	Reuso do perfil; dados bancários separados por permissão
Entregas	Portal (aprovação), Time (semana de entregas), Performance, Resultados	Estado calculado por <code>entrega-engine</code>
Comercial	Clientes, Projetos, E-mail marketing (leads), Webhooks de saída, Calculadora pública	Conversão, <code>lead.created</code> / <code>lead.won</code> , token de proposta
Projetos/Tarefas	Chat (@tarefa/@projeto), Time, Início (Meu trabalho), Problemas (bugs de projeto), Metas	<code>task-aggregation</code> junta projeto + campanha + marketing
Reuniões	Google Calendar, Chat (chamada), Início (agenda), lembretes, push	OAuth + cron diário
Financeiro	Contratos, Campanhas, Influenciadores, Clientes	<code>clienteId/campanhaId/influenciadorId</code> no lançamento
Metas	Independente dos demais módulos (indicadores manuais; <code>dataSource: auto reservado</code>)	<code>metas-engine</code>
Time	Chat (tempo de resposta), Tarefas, Entregas, Configurações (score)	<code>performance-engine</code> , <code>get_member_response_time</code>
Chat	Todos (menções), Push, Início, Notificações	<code>mention-kinds</code> : pessoa, tarefa, projeto, campanha, cliente
Central de Problemas	Portal do cliente (reporta), Projetos (bugs), links <code>/bugs/\$token</code> , AppShell (botão), Hypito	Mesma tabela <code>bug_reports</code>
Notificações	Realtime de leads, ações do cliente, demandas, mensagens, release	Sino do AppShell + toast + push
Configurações	Permissões de todos os módulos, Integrações, Segurança, Score	<code>profiles.permissions</code> , <code>outgoing_webhooks</code> , <code>webhook_settings</code>
E-mail marketing	Leads, Clientes, Influenciadores (públicos), Resend (envio/webhook), Cron (fluxos), descadastro	<code>email_*</code> , <code>unsubscribe_token</code>
Blog	Webhook para o site (Make), Portal (cliente curte/comenta artigos)	<code>blog_*</code> , <code>BLOG_WEBHOOK_URL</code>
Hypito	Chat (<code>hypito_payload</code>), lembretes, briefing diário, relatórios, alertas, releases	Tabelas <code>hypito_*</code> [inferido: serviço externo]

4. Fluxos ponta a ponta

A. Cliente aprova um influenciador Portal V2 → `respondCampanhaInfluSession` → `campanha_influenciadores.data` (inclui `lastClientAction`) → Realtime UPDATE → AppShell mostra item no sino e atualiza o card da campanha; o portal também reflete via seu próprio canal.

B. Cliente envia uma demanda Portal → `submitClientDemandSession` → insere em `campanha_tarefas` → Realtime INSERT → notificação + toast na equipe, e a tarefa entra em "Meu trabalho".

C. Lead chega de um formulário Typeform/Make → POST `/api/public/leads` (segredo) → `leads` → Realtime → pipeline atualiza e AppShell notifica. Ao ganhar/criar pela UI, `lead.won` / `lead.created` saem para os webhooks configurados.

D. Inscrição de influenciador Link `/inscricao/$token` → `submitInscricaoCampanha` (idempotente) → entra na campanha como "inscrito" → aparece na curadoria do Banco/Board → pode ser enviado ao cliente.

E. E-mail marketing Admin cria campanha de e-mail → `activateCampaign` → cron diário avança fluxos → Resend envia → webhook Svix atualiza `email_sends` → descadastro (`unsubscribe_token`) cancela o destinatário em todas as campanhas.

F. Reunião ↔ Google Conexão OAuth (`startGoogleOAuth` → `callback`) → sync manual (`syncAllMeetingsToGoogle` / `importGoogleEventsToMeetings`) e cron `google-calendar-sync` diário.

G. Convite de pessoa do cliente Admin do cliente (Portal → Configurações → Acessos) ou equipe (Cliente → Acessos ao portal) → server function de convite (link gerado por `auth.admin.generateLink`, enviado por `sendEmail` /Resend) → `/criar-senha` → login → `acceptPendingInvites` → ambiente `client` → `/portal-v2/inicio`. Gate de NPS pode bloquear até responder.

H. Release `scripts/release.ts` publish → `platform_releases` → Realtime → `VersionWatcher` /release notes avisam; `public/version.json` sem cache indica bundle novo.

5. Onde a comunicação é frágil (a observar)

- Muita regra depende de **JSONB** (`clientes.data.campanhas` , `*.data`): vínculos não têm chave estrangeira; consistência é por convenção e triggers (`sync_scoped_table_data_id`).
- **Realtime sem fila**: se a aba estiver fechada, a notificação só chega via push ou ao recarregar.
- **Dois chats** (V1 e V2) e **três gerações de portal** convivem; mudanças precisam considerar ambos.
- **Webhooks de saída são best-effort**: falha é só logada.

07 · Design

O contrato completo é `../design-system/DESIGN-SYSTEM.md` (regras), com decisões em `DECISIONS.md`, evidências em `AUDIT-FINDINGS.md` e a página Início como referência de linguagem em `INICIO-REFERENCE.md`. Aqui está o resumo operacional e **como cada superfície aplica**.

Princípios

Hierarquia por tipografia e espaço, não por cor. Cor só para ação, seleção e estado. Superfícies planas (sem sombra), densidade alta. Arquitetura de UX vale tanto quanto o visual: página única por contexto, contexto declarado uma vez, **um só mecanismo de filtro**, uma ação primária por contexto, um só fluxo de rolagem, mesma função = mesmo componente.

Tokens (`src/styles.css` , `src/lib/design-tokens.ts`)

Grupo	Tokens
Superfície	<code>background</code> , <code>card</code> , <code>popover</code> , <code>muted</code> ; card elevado <code>surface-card</code> (= <code>SURFACE.card</code>)
Texto	<code>foreground</code> ; secundário <code>text-secondary</code> (= <code>muted-foreground</code> , alinhados); marca <code>text-brand</code> (AA no claro)
Marca/ação	<code>brand</code> , <code>brand-hover</code> , <code>brand-subtle</code> , <code>brand-foreground</code> , <code>brand-foreground-secondary</code> , <code>brand-border</code>
Estado	<code>success</code> , <code>warning</code> , <code>danger</code> , <code>info</code> (+ <code>-soft</code> , <code>-soft-foreground</code> , <code>-border</code>), <code>destructive</code>
Borda/foco	<code>border</code> , <code>input</code> , <code>ring</code>
Raio	controle <code>rounded-md</code> · linha/tile <code>rounded-lg</code> · card e overlay <code>rounded-2xl</code> · pill <code>rounded-full</code>
Tipografia	pilha do sistema; escala em <code>TYPGRAPHY</code> : pageHeading 24/26 600, section/card 15 600, body 14, metadata 11, label caixa-alta 11, KPI 20/24 600, KPI lead 30/36 700
Espaço	múltiplos de 4px; blocos 24→32, cards 16→24
Temas claro e escuro, todos em <code>oklch</code> .	

Componentes canônicos

`PageHeader` · `Card/CardHeader` (`ui/card`) · `Kpi` (Strip e Lead) · `FilterToolbar` (busca, Filtros, Ordenar, chips) · `SegmentedControl` · `Tabs` (só em overlay) · `Badge` · `EmptyState` · `Alert` · `Skeleton` · `NativeSelect` · `Table` (scroll horizontal no container) · `Dialog` · `Sheet` (drawer) · `useConfirm` . Vitrine em `/design-system` .

Arquitetura de página (app interno)

Cabeçalho → contexto global (período, mês) → KPIs (só se decidem algo) → Busca + Filtros → conteúdo. Detalhe: retorno → identidade → KPIs curtos → seções na mesma página com âncora. Shell: sidebar 256/68px, topbar 64px, `main p-4 md:p-8` , `PageContainer` 1280/1600/total.

Estado da migração

Item	Situação
Título oficial, sem canvas cinza, cards <code>surface-card</code> , tokens de texto e marca, tipografia 9/10/13px/bold/light, selects nativos, confirmação via <code>useConfirm</code>	Aplicado na plataforma (commit <code>f8679aa</code>), validado visualmente só em Cientes
KPI, filtros e estados vazios dos demais módulos; caixas de erro; pills e abas feitas à mão; cores de estado por paleta direta; tabelas cruas; Radix Select (5 arquivos)	Pendente
Cores de etapa do pipeline (decisão P4: neutras + semântica real)	Decidido, não implementado
Alvos de toque de 44px nos filtros	Pendente
Validação visual dos demais módulos, especialmente telas densas e escuro	Pendente (precisa de login)

Portal do cliente

Mesma base de tokens e a mesma superfície elevada do Início (`SURFACE.raised` , `rounded-2xl`), com decisões próprias: - **Linguagem sem alarme**: o teste `no-risk-language` garante que os derivadores do portal V2 (atenção, resumo de campanhas, atividade) não vazem rótulos internos de risco ou atraso ("em risco", "atrasado", "crítico"...); status do cliente usa rótulos neutros em `client-status` . - **Foco em decisão**: a Início do portal destaca "o que espera você" (`ClientAttentionList`), progresso das campanhas e atividade. - **Formatadores próprios** em `features/client-portal-v2/lib` (`metric-format` , `competencia` , `ajuste-format` , `client-file-format`) para nunca expor dado cru. - Layout `PortalV2Shell` (sidebar Início/Campanhas/Relatórios/Arquivos, perfil, tema) e **NPS como porta**: bloqueia a navegação até responder. - Portal por token usa componentes próprios (`components/portal/*`) e `portal-i18n` ; ainda não passou pela migração de cards/tipografia — **a verificar**.

Páginas públicas e e-mail

Inscrição, NPS do influenciador, bugs, proposta e descadastro são páginas **fora do shell**, centradas, com logo/nome do workspace (`workspace-store`) e `noindex` . A inscrição tem página de edição por campanha (`InscricaoPageDialog`) com pré-visualização. E-mails usam `email-template` + templates editáveis (`email_templates`).

Acessibilidade e responsividade (regras do contrato)

Foco visível; botão só de ícone com nome acessível; linha clicável com `role="button"` ; contraste $\geq 4.5:1$ nos dois temas; `prefers-reduced-motion` ; mobile-first, sidebar vira drawer abaixo de `md` ; a página nunca rola na horizontal (só tabelas, dentro do container).

08 · Lacunas, pendências e riscos

O que não foi verificado neste levantamento

- **Dados reais:** contagens, volume e qualidade dos registros em produção não foram consultados.
- **Telas em produção:** o app exige login; nenhuma tela autenticada foi aberta. O visual só foi validado em harness isolado (Clientes).
- **Hypito:** o serviço não está neste repositório; seu comportamento é deduzido de tabelas, permissões e `hypito_payload`.
- **Políticas RLS linha a linha:** documentamos o desenho (funções e permissões), não auditamos cada policy. Ver `security-audit-report.md`.
- **Funções de servidor:** listadas por arquivo/nome; o corpo de cada uma não foi revisado.
- **Portal por token:** o conteúdo exato de cada tela não foi lido em detalhe.

Pendências conhecidas (do projeto)

- `membros` e `senhas` são permissões **decorativas** (ver 03).
- `clientes` aceita permissão `campanhas` por causa do JSONB.
- Segredo do webhook de leads vive em tabela (não em env); `CLAUDE.md` ainda cita `LEADS_WEBHOOK_SECRET` como legado.
- Chat V1 e V2, e portal V2 + token + legado, convivem.
- Metas: indicadores automáticos (`dataSource: auto`) reservados, sem implementação.
- `SummaryStat`, `MetricCard` (só na vitrine) e `PipelineSummary` ainda não convergem para o KPI canônico.
- Arquivos não rastreados soltos no repositório: `.claude/scheduled_tasks 2.lock`, `src/lib/portal-auth.functions 2.ts` (cópia duplicada — deve ser revisada e removida).
- `.claude/worktrees/*` são cópias antigas que quebram o `vitest` padrão; excluir da execução ou apagar.

Riscos técnicos

Risco	Efeito	Mitigação sugerida
Vínculos em JSONB sem FK	Inconsistência silenciosa (campanha órfã, id divergente)	Triggers existentes + testes; considerar normalizar <code>campanhas</code>
Realtime sem fila	Notificação perdida com aba fechada	Push e recarga já cobrem parte; monitorar
Sem CI	Regressão passa até o deploy	Rodar <code>typecheck</code> , <code>lint</code> , <code>test</code> , <code>build</code> antes de cada push
Arquivos enormes (<code>CampanhasSection</code> ~1.700 linhas; <code>TaskBoard</code> , <code>InfluencerBoard</code> > 4.000)	Difícil manter e testar	Quebrar por responsabilidade
Service-role em server functions	Bypass de RLS se faltar checagem	Sweep já feito; manter <code>assertAdmin</code> /validação de token em toda função nova
Webhooks de saída best-effort	Evento perdido sem aviso	Considerar log/retry
Migração visual sem validação autenticada	Contraste ou separação ruins em telas densas	Validar módulo a módulo logado

Próximos passos sugeridos

1. Validar visualmente os módulos migrados (claro e escuro, mobile).
2. Converter KPI, filtros e estados vazios restantes ao padrão canônico.
3. Implementar a decisão P4 (pipeline neutro) e P3 (tabelas com scroll no container).
4. Remover `portal-auth.functions 2.ts` e os `worktrees` antigos.
5. Reconciliar `CLAUDE.md` (variáveis e permissões) com o que está aqui.